LECTURE

React,

from zero to useful.

Node, npm, Vite, components, props, state, events, lists, and forms for building interactive web apps.

tools components state events forms

THE TARGET

By the end, React should feel runnable.

STUDENTS SHOULD BE ABLE TO

- ✓ Open and run a Vite React project.
- ✓ Explain why Node and npm are involved.
- ✓ Edit `App.jsx` and see the page change.
- ✓ Create small components and reuse them.

AND BUILD WITH

- ✓ Props for parent-to-child data.
- ✓ State for data that changes.
- ✓ Events for clicks, typing, and submits.
- ✓ Lists and forms for real app behavior.

not mastery today. enough confidence to start.

NEW TOOLCHAIN

We are not just opening an HTML file anymore.

React projects use tools that run before the browser receives the final JavaScript.

the browser still shows the app. Vite prepares it.

THE ENGINE

Node lets JavaScript run on your computer.

Before this, most of our JavaScript ran inside the browser.

Node lets JavaScript run as a normal computer program, which means it can power developer tools.

Today, Node is mainly running the tools that build and serve our React app.

SAY IT SIMPLY

Node is the engine under the hood. You do not write your UI in Node today. You use Node to run the project tools.

PACKAGES + SCRIPTS

npm manages the code your project depends on.

PACKAGE

A package is code someone else wrote that your project can use. React and Vite are packages.

PACKAGE.JSON

This file is the project's ingredient list. It records dependencies and useful commands.

```
npm install       # download everything the project needs
npm run dev       # start the dev server script
```

LIVE TRANSLATOR

Vite sits between React code and the browser.

1. TRANSLATES

JSX becomes regular JavaScript the browser understands.

2. SERVES

It runs a local website, usually on `localhost`.

3. REFRESHES

When files change, the browser updates quickly.

Vite is a live translator during development.

WHAT DID VITE MAKE?

A React project is a folder with a few jobs.

```
my-react-app/
  index.html
  package.json
  vite.config.js
  src/
    main.jsx
    App.jsx
    index.css
```

TODAY'S FOCUS

- ✓ `App.jsx`: first component to edit.
- ✓ `main.jsx`: attaches React to HTML.
- ✓ `index.css`: app styling.
- ✓ `package.json`: commands and packages.

HOW THE PAGE STARTS

One HTML page loads one React app.

```html
// index.html
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

```jsx
// src/main.jsx
import React from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(<App />)
```

main.jsx is the starting line. App.jsx is where we begin building.

THE PAIN

Vanilla DOM work does not scale kindly.

MANUAL MODE

- × Track data in loose variables.
- × Find elements with selectors.
- × Manually update text, classes, and lists.
- × Remember every place that must change.

REACT MODE

- ✓ Put changing data in state.
- ✓ Describe the UI for that state.
- ✓ React updates the DOM for you.
- ✓ Events change state, not random HTML.

DECLARATIVE UI

You describe the result. React handles the DOM.

Instead of saying, "go find this element and change it," we say, "given this data, the screen should look like this."

events update state, then React renders again.

WHO DOES WHAT?

These names each have one job.

REACT

Library for building UI with components.

JSX

HTML-looking syntax written inside JavaScript.

VITE

Dev server, JSX translation, and bundling.

NODE

Runs project tools on your computer.

NPM

Installs packages and runs scripts.

BROWSER

Runs the final JavaScript and displays the UI.

You do not configure Babel today. Vite handles JSX translation for us.

START SIMPLE

A component is a function that returns JSX.

```
function Hello() {
  return <h1>Hello, React!</h1>
}

export default Hello
```

WHAT TO NOTICE

- ✓ It is a JavaScript function.
- ✓ The function returns UI.
- ✓ The UI looks like HTML.
- ✓ The component name is capitalized.

JSX RULES

JSX looks like HTML, but follows JavaScript rules.

RULES STUDENTS HIT FIRST

- ✓ Return one outer element or a fragment.
- ✓ Use {} to insert JavaScript values.
- ✓ Use `className`, not `class`.
- ✓ Close every tag, including `<input />`.

```jsx
const name = 'Maya'

return (
  <>
    <h1 className="title">Hi, {name}</h1>
    <input placeholder="Type here" />
  </>
)
```

REUSABLE UI RECIPES

Components let us make our own tags.

```
function Greeting() {
  return <h2>Welcome!</h2>
}

function App() {
  return (
    <main>
      <Greeting />
      <Greeting />
    </main>
  )
}
```

CAPITALIZATION RULE

- ✓ is treated like HTML.
- ✓ <Greeting /> is treated like your component.
- ✓ Lowercase component names confuse React.

PARENT TO CHILD

Props are information passed into a component.

```
function Greeting(props) {
  return <h2>Hi, {props.name}!</h2>
}

function App() {
  return (
    <>
      <Greeting name="Sarah" />
      <Greeting name="Marcus" />
    </>
  )
}
```

MENTAL MODEL

- ✓ Props work like attributes.
- ✓ The parent chooses the values.
- ✓ The child receives a props object.
- ✓ Props are read-only in the child.

SHORTCUT

Destructuring just unpacks the props object.

LONG VERSION

```
function Greeting(props) {
  return <h2>Hi, {props.name}!</h2>
}
```

SHORTCUT VERSION

```
function Greeting({ name }) {
  return <h2>Hi, {name}!</h2>
}
```

same data. less typing. not React magic.

USESTATE

State is data a component remembers.

When state changes, React runs the component again and updates the screen to match.

```
import { useState } from 'react'

function Counter() {
  const [count, setCount] = useState(0)
  return <p>Count: {count}</p>
}
```

THE SETTER MATTERS

Do not change state directly.

CORRECT

```
const addOne = () => {
  setCount(count + 1)
}
```

BUG

```
const addOne = () => {
  count = count + 1
}
```

The setter is how you tell React, ”this value changed; please render again.”

regular assignment changes a variable. it does not notify React.

INDEPENDENT INSTANCES

Same component. Different memory.

```
<Counter label="Apples" />
<Counter label="Mangos" />
<Counter label="Peaches" />
```

WHAT TO PICTURE

- ✓ Three component instances render.
- ✓ Each instance has its own count.
- ✓ Clicking one does not update the others.
- ✓ Shared truth must move up to a parent.

REACT EVENTS

Pass a function. Do not call it early.

CORRECT

```
<button onClick={addOne}>
  Add one
</button>
```

COMMON BUG

```
<button onClick={addOne()}>
  Add one
</button>
```

The first version says: ”React, run this later when the click happens.” The second version runs immediately during render.

WRAP THE CALL

Inline arrow functions run later too.

```
<button onClick={() => setCount(count + 1)}>
  Add one
</button>

<button onClick={() => sayHello('Ava')}>
  Say hello
</button>
```

WHY THE WRAPPER?

- ✓ The click needs a function to run later.
- ✓ () => creates that function.
- ✓ Useful when you need to pass an argument.
- ✓ Good for simple one-liners.

MAP + KEY

Arrays become UI with map().

```
const fruits = ['Apple', 'Mango', 'Peach']

return (
  <ul>
    {fruits.map((fruit) => (
      <li key={fruit}>{fruit}</li>
    ))}
  </ul>
)
```

KEYS

- ✓ React compares old list to new list.
- ✓ Keys identify each item.
- ✓ Use a stable unique value.
- ✓ Real data usually has an `id`.

DATA DOWN, EVENTS UP

A child can call a function from its parent.

```
function App() {
  const [items, setItems] = useState([])
  const addItem = (text) => setItems([...items, text])

  return <AddButton onAdd={addItem} />
}

function AddButton({ onAdd }) {
  return <button onClick={() => onAdd('New item')}>Add</button>
}
```

the parent owns the state. the child reports what happened.

BRIDGE

You can display data now. Forms let users send data back.

Forms combine everything: JSX, state, events, props, validation, and the browser’s default behavior.

START WITH HTML

A form groups inputs and fires submit.

```
function NewPost() {
  return (
    <form>
      <input name="title" />
      <button type="submit">Create</button>
    </form>
  )
}
```

RENDERED IDEA

Title

Create

BROWSER DEFAULT

Forms try to refresh the page.

```
function NewPost() {
  const handleSubmit = (e) => {
    e.preventDefault()
    console.log(e.target.title.value)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="title" />
    </form>
  )
}
```

FIRST FORM RULE

- ✓ Submit gives us an event.
- ✓ `preventDefault()` stops reload.
- ✓ `name` lets us find the input.
- ✓ This is useful before controlled inputs.

REACT OWNS VALUE

value displays state. onChange updates it.

```
const [title, setTitle] = useState('')

<input
  value={title}
  onChange={(e) => setTitle(e.target.value)}
/>
```

1. USER TYPES

Browser fires change.

2. SETTER RUNS

State receives next value.

3. REACT RENDERS

Input displays state.

SCALE SLOWLY

Use separate state first. Then show object state.

BEGINNER-FRIENDLY FIRST

```
const [title, setTitle] = useState('')
const [content, setContent] = useState('')
```

SCALING PATTERN LATER

```
const [form, setForm] = useState({ title: '', content: '' })

setForm({
  ...form,
  [e.target.name]: e.target.value
})
```

spread keeps the old fields. name chooses which field changes.

SUMMARY

React is tools, components, state, and events.

TOOLS

- ✓ Node runs the tooling.
- ✓ npm installs packages and runs scripts.
- ✓ Vite translates JSX and serves the app.
- ✓ The browser still displays the final UI.

COMPONENTS

- ✓ Components are functions that return JSX.
- ✓ Props pass data from parent to child.
- ✓ Lists come from mapping arrays.
- ✓ Keys help React track list items.

STATE + EVENTS

- ✓ State remembers changing data.
- ✓ Setters tell React to render again.
- ✓ Events update state.
- ✓ Forms are state plus submit/change events.

thinking in UI means: choose the state, render from it, update it with events.